SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT 11

1. Student Details

Student Name	Swanit Bhawar
PRN	202501110203
Division / Batch	C / C2
Roll No.	202501110203
Date of Submission	02-11-2026

2. Practical Details

Practical No.	11(a)
Practical Title	STL - List Container
Date Performed	29-10-2026
Faculty Name	Ms Shruti

3. Problem Statement

Implement the C++ Standard Template Library list container and demonstrate insertion, deletion, sorting, removal of duplicate values, searching, iterator-based insertion and erasure, splicing, reverse traversal and clearing of elements.

4. Objectives

1. Understand the STL list container and its iterators.
2. Perform list insertion, removal, sorting, searching and reversing.
3. Demonstrate splice, erase and clear operations with appropriate output.

5. Expected Learning Outcomes

1. Create and traverse a list of integers.
2. Apply STL list member functions correctly.
3. Differentiate iterator-based operations from index-based array access.

6. Theory / Concept

The Standard Template Library provides reusable containers, iterators and algorithms. **std::list** is a doubly linked list supporting efficient insertion and removal at known iterator positions. Unlike a vector, it does not support random access through indices. Its member functions include push_back(), push_front(), pop_back(), pop_front(), sort(), unique(), reverse(), remove(), splice(), erase() and clear(). The find() algorithm returns an iterator to a matching value, or end() if absent.

7. Algorithm / Pseudocode

1. Create an empty list of integers.
2. Insert values at the front and back; display the size, first element and last element.
3. Remove elements from both ends; add duplicates; sort, remove adjacent duplicates and reverse.
4. Find 30 and insert 35 before it; remove 10 and splice another list.
5. Sort again, erase the third element, traverse in reverse and clear the list.

8. Technology / Tools / Libraries Used

Programming Language	C++
IDE / Platform	Visual Studio Code / Windows
Libraries	iostream, list, algorithm, iterator, string
Compiler	g++

9. Implementation / GitHub Repository

Repository Name	Swanit95/Object-Oriented-Programming-
Repository URL	https://github.com/Swanit95/Object-Oriented-Programming-
Assignment Folder	assignments/Assignment 11
Branch	main
Commit ID	30d881aa7b3ef6f95ce4d1dc8f96dd2ab4f665fe

Implementation

STL List Container

STL_list_container.cpp

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to display list elements
void show(const list<int>& l, const string& label) {
    cout << label << " ";
    for (int x : l) {
        cout << x << " ";
    }
    cout << endl;
}

int main() {
    list<int> l;

    // Insertion at both ends
    l.push_back(30);
    l.push_back(10);
    l.push_back(20);
    l.push_front(50);

    show(l, "After insertion:");
    cout << "Size: " << l.size() << endl;
    cout << "Front: " << l.front()
         << ", Back: " << l.back() << endl;

    // Removing elements from both ends
    l.pop_front();
    l.pop_back();
    show(l, "After pop_front and pop_back:");

    // Adding duplicate values
    l.push_back(10);
    l.push_back(40);
    l.push_back(10);

    show(l, "With duplicates:");

    // Sorting the list
    l.sort();
    show(l, "After sort:");

    // Removing consecutive duplicates
    l.unique();
    show(l, "After unique:");

    // Reversing the list
    l.reverse();
    show(l, "After reverse:");

    // Searching and inserting an element
    list<int>::iterator it =
        find(l.begin(), l.end(), 30);

    if (it != l.end()) {
        l.insert(it, 35);
    }

    show(l, "After insert 35 before 30:");

    // Removing an element by value
    l.remove(10);
    show(l, "After remove(10):");
```

```cpp
    // Sorting the list
    l.sort();
    show(l, "After sort:");

    // Removing consecutive duplicates
    l.unique();
    show(l, "After unique:");

    // Reversing the list
    l.reverse();
    show(l, "After reverse:");

    // Searching and inserting an element
    list<int>::iterator it =
        find(l.begin(), l.end(), 30);

    if (it != l.end()) {
        l.insert(it, 35);
    }

    show(l, "After insert 35 before 30:");

    // Removing an element by value
    l.remove(10);
    show(l, "After remove(10):");

    // Splicing another list
    list<int> l2 = {5, 15};
    l.splice(l.end(), l2);

    show(l, "After splice:");
    cout << "l2 size after splice: "
         << l2.size() << endl;

    // Sorting again
    l.sort();
    show(l, "After sort:");

    // Erasing the third element
    it = l.begin();
    advance(it, 2);
    l.erase(it);

    show(l, "After erase 3rd element:");

    // Reverse traversal using reverse iterators
    cout << "Reverse traversal: ";
    for (auto r = l.rbegin(); r != l.rend(); ++r) {
        cout << *r << " ";
    }
    cout << endl;

    // Clearing the list
    l.clear();

    cout << "After clear, empty? "
         << (l.empty() ? "Yes" : "No") << endl;

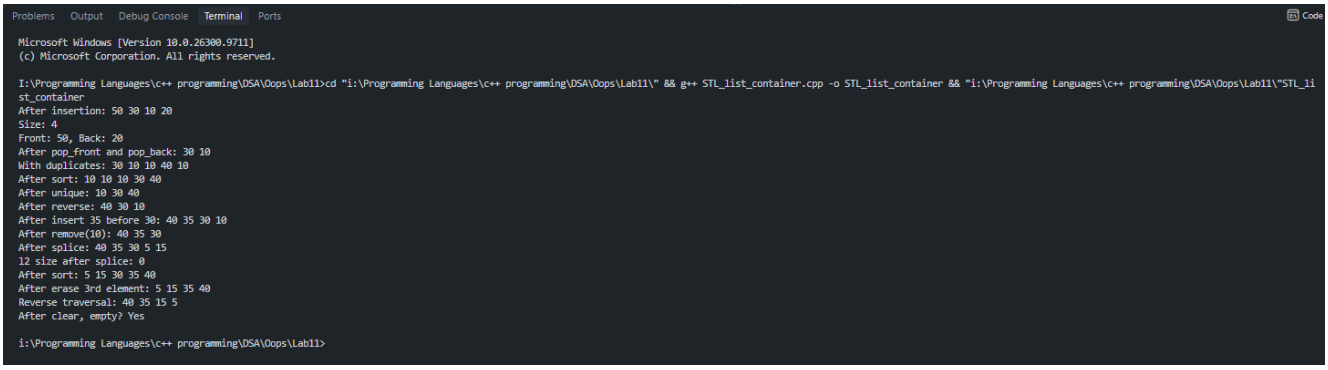
    return 0;
}
```

10. Test Cases / Input and Output

Test Case	Expected Result	Actual Result	Status
Insertion / removal	50 30 10 20; then 30 10	Matched	Pass
Sort / unique / reverse	10 30 40; then 40 30 10	Matched	Pass
Insert / splice / erase	Correct list order	Matched	Pass
Clear	Empty? Yes	Empty? Yes	Pass

11. Results / Observations

The program compiled and executed successfully. Sorting, duplicate removal, searching and insertion, splicing, iterator-based erasure, reverse traversal and clearing produced the expected output.



12. Conclusion

The practical demonstrated the STL list container and its member functions, with correct operations on integers and appropriate use of iterators.

13. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Swanit Bhawar	202501110203	